

UFSM00281 - PROJETO DIGITAL COM FPGA

Controlador Semafórico em FPGA

Angelo Luigi Bocchi Lovatto - 202220528

26 de novembro de 2025

1 Objetivo

Desenvolvi um controlador de tráfego para um cruzamento de duas vias (Rua Principal e Rua Secundária) utilizando a placa FPGA Nexys 3 (Spartan-6). O sistema deve operar com uma Máquina de Estados Finita (FSM) para garantir a segurança viária, incluir temporizadores visuais, permitir interrupção inteligente por pedestres e possuir um modo noturno de emergência.

2 Especificação do Sistema

2.1 Entradas e Saídas

- **Entradas:**
 - `clk` (100 MHz): Clock do sistema.
 - `sw_night` (SW0): Ativa o Modo Noturno (Amarelo Piscante).
 - `btn_ped` (Btn Central): Botão de solicitação de travessia de pedestre.
- **Saídas Visuais (LEDs):**
 - **Rua Principal (Esq):** LED 7 (Vermelho), LED 6 (Amarelo), LED 5 (Verde).
 - **Rua Secundária (Dir):** LED 2 (Vermelho), LED 1 (Amarelo), LED 0 (Verde).
- **Display 7 Segmentos:** Exibe a contagem regressiva (em segundos) para a próxima mudança de sinal.

3 Diagrama de Estados (FSM)

A máquina de estados controla o ciclo normal de operação. O sistema possui 5 estados principais dispostos em um ciclo fechado.

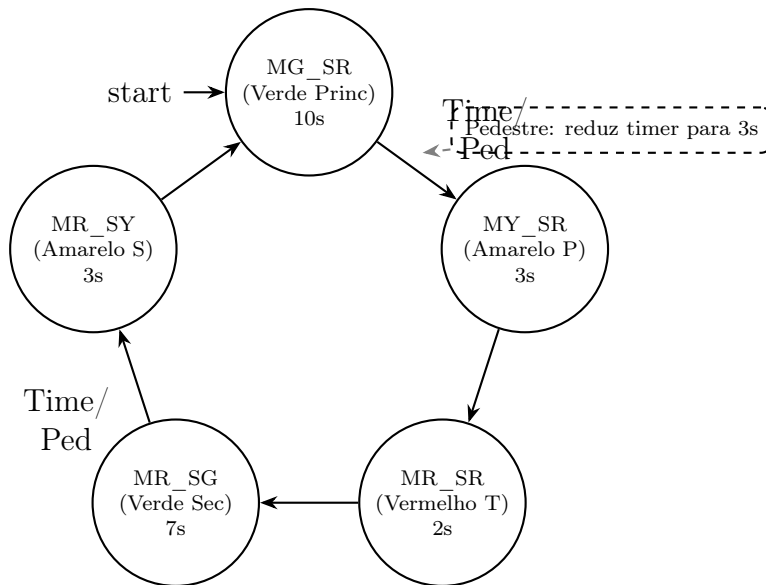


Figura 1: Ciclo da Máquina de Estados do Semáforo.

4 Manual de Operação

Para melhor entendimento da apresentação do projeto, aqui está a explicação de cada momento

4.1 1. Visualização na Placa

Os LEDs foram mapeados para simular a posição física dos semáforos:

- **LEDs 7, 6, 5 (Esquerda):** Semáforo da Rua Principal.
- **LEDs 2, 1, 0 (Direita):** Semáforo da Rua Secundária.
- **Display:** Mostra o tempo restante do sinal Verde atual.

4.2 2. Cenário A: Ciclo Normal

Com o **Switch 0 desligado**, o sistema opera ciclicamente:

1. **Verde Principal (10s):** Carros passam na via principal.
2. **Amarelo Principal (3s):** Atenção para parada.
3. **Vermelho Geral (2s):** Segurança (ambos fechados) para evitar colisões.
4. **Verde Secundária (7s):** Fluxo na via menor.
5. **Amarelo Secundária (3s):** Atenção.
6. Retorna ao início.

4.3 3. Cenário B: Pedestre Inteligente

Quando o sinal está Verde (para qualquer rua) e o contador está alto (ex: 9, 8, 7...), se um pedestre pressionar o **Botão Central**:

- O sistema detecta a solicitação.
- O tempo restante é forçado para **3 segundos** imediatamente.
- O sinal fecha mais rápido, priorizando a travessia segura sem cortes abruptos (passa pelo amarelo).

4.4 4. Cenário C: Modo Noturno/Emergência

Ao ligar o **Switch 0 (SW0)**:

- A FSM é ignorada.
- **Rua Principal:** Pisca em Amarelo (Atenção).
- **Rua Secundária:** Pisca em Vermelho (Pare).
- **Display:** Mostra um traço (-) indicando temporizador inativo.

5 Implementação em VHDL

5.1 Top Module (semaforo.vhd)

Este código integra a FSM, o temporizador, a lógica do pedestre e o controle de LEDs. A lógica foi unificada em processos para evitar conflitos de *drivers*.

Listing 1: Código principal do Controlador Semafórico.

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.NUMERIC_STD.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity semaforo is
7     Port ( clk      : in  STD_LOGIC;
8           sw_night  : in  STD_LOGIC;
9           btn_ped   : in  STD_LOGIC;
10          leds      : out STD_LOGIC_VECTOR (7 downto 0);
11          seg       : out STD_LOGIC_VECTOR (7 downto 0);
12          an        : out STD_LOGIC_VECTOR (3 downto 0));
13 end semaforo;
14
15 architecture Behavioral of semaforo is
16     -- Componente Debounce omitido por brevidade --
17     component debounce
18     Port ( clk : in STD_LOGIC; btn_in : in STD_LOGIC; btn_out : out STD_LOGIC);
19     end component;
20
21     signal btn_ped_clean, btn_ped_pulse, btn_prev : STD_LOGIC := '0';
22
23     -- Estados: Main Green/Side Red, etc.
24     type state_type is (MG_SR, MY_SR, MR_SR, MR_SG, MR_SY);
25     signal state : state_type := MG_SR;
26     signal next_state : state_type;
27
28     constant ONE_SEC : integer := 100000000;
29     signal timer_tick : integer range 0 to ONE_SEC := 0;
```

```

30 signal countdown : integer range 0 to 15 := 10;
31 signal blink_active : std_logic := '0'; -- Para o modo noturno
32
33 -- Sinais intermediarios de cor
34 signal led_main_fsm, led_side_fsm : STD_LOGIC_VECTOR(2 downto 0);
35 signal led_main_final, led_side_final : STD_LOGIC_VECTOR(2 downto 0);
36
37 -- Display
38 signal digit_val : integer range 0 to 15;
39 signal active_digit : integer range 0 to 3 := 0;
40 signal seg_temp : STD_LOGIC_VECTOR(7 downto 0);
41 signal an_temp : STD_LOGIC_VECTOR(3 downto 0);
42
43 begin
44     -- MULTIPLEXADOR DE SAIDA (Prioridade: Modo Noturno > FSM)
45     led_main_final <= "010" when (sw_night='1' and blink_active='1') else
46         "000" when (sw_night='1' and blink_active='0') else
47         led_main_fsm;
48
49     led_side_final <= "100" when (sw_night='1' and blink_active='1') else
50         "000" when (sw_night='1' and blink_active='0') else
51         led_side_fsm;
52
53     leds(7 downto 5) <= led_main_final;
54     leds(4 downto 3) <= "00";
55     leds(2 downto 0) <= led_side_final;
56
57     DB_PED: debounce port map(clk, btn_ped, btn_ped_clean);
58
59     -- DETECTOR DE BORDA DO BOTAO
60     process(clk)
61     begin
62         if rising_edge(clk) then
63             btn_prev <= btn_ped_clean;
64             btn_ped_pulse <= btn_ped_clean and (not btn_prev);
65         end if;
66     end process;
67
68     -- LOGICA DE TEMPO E ESTADOS
69     process(clk)
70     begin
71         if rising_edge(clk) then
72             -- Gerador de Blink (0.5s)
73             if timer_tick > (ONE_SEC / 2) then blink_active <= '1'; else blink_active <=
74                 '0'; end if;
75
76             if sw_night = '0' then
77                 if timer_tick = ONE_SEC then
78                     timer_tick <= 0;
79                     if countdown > 0 then
80                         countdown <= countdown - 1;
81                     else
82                         state <= next_state; -- Muda de Estado
83                         case next_state is
84                             when MG_SR => countdown <= 9;
85                             when MY_SR => countdown <= 2;
86                             when MR_SR => countdown <= 1;
87                             when MR_SG => countdown <= 6;
88                             when MR_SY => countdown <= 2;
89                             when others => countdown <= 0;
90                         end case;
91                     end if;
92                 else
93                     timer_tick <= timer_tick + 1;
94                     -- Interrup o de Pedestre (Encurta o tempo)
95                     if btn_ped_pulse = '1' then
96                         if (state = MG_SR or state = MR_SG) and countdown > 3 then
97                             countdown <= 3;
98                         end if;
99                     end if;
100                 end if;
101             else
102                 state <= MG_SR; -- Reseta FSM no modo noturno

```

```

102         countdown <= 9;
103         timer_tick <= timer_tick + 1;
104     end if;
105 end if;
106 end process;
107
108 -- DECODIFICADOR DE ESTADOS
109 process(state)
110 begin
111     case state is
112         when MG_SR => led_main_fsm<="001"; led_side_fsm<="100"; next_state<=MY_SR;
113         when MY_SR => led_main_fsm<="010"; led_side_fsm<="100"; next_state<=MR_SR;
114         when MR_SR => led_main_fsm<="100"; led_side_fsm<="100"; next_state<=MR_SG;
115         when MR_SG => led_main_fsm<="100"; led_side_fsm<="001"; next_state<=MR_SY;
116         when MR_SY => led_main_fsm<="100"; led_side_fsm<="010"; next_state<=MG_SR;
117         when others => led_main_fsm<="100"; led_side_fsm<="100"; next_state<=MG_SR;
118     end case;
119 end process;
120
121 -- DRIVER DO DISPLAY (Omitido detalhes repetitivos de mux e 7seg para brevidade) --
122 process(active_digit, countdown, sw_night)
123 begin
124     an_temp <= "1111"; digit_val <= 0;
125     if sw_night = '1' then digit_val <= 10; -- Tra o
126     else
127         case active_digit is
128             when 0 => an_temp <= "1110"; digit_val <= countdown mod 10;
129             when 1 => an_temp <= "1101"; digit_val <= countdown / 10;
130             when others => an_temp <= "1111";
131         end case;
132     end if;
133 end process;
134 -- (Logica de atualizacao de anodos e decodificacao de segmentos padrao) --
135 process(digit_val)
136 begin
137     case digit_val is
138         -- 0 a 9 padrao ...
139         when 0 => seg_temp <= "11000000";
140         when 1 => seg_temp <= "11111001";
141         when 2 => seg_temp <= "10100100";
142         when 3 => seg_temp <= "10110000";
143         when 4 => seg_temp <= "10011001";
144         when 5 => seg_temp <= "10010010";
145         when 6 => seg_temp <= "10000010";
146         when 7 => seg_temp <= "11111000";
147         when 8 => seg_temp <= "10000000";
148         when 9 => seg_temp <= "10010000";
149         when 10 => seg_temp <= "10111111"; -- Tra o
150         when others => seg_temp <= "11111111";
151     end case;
152 end process;
153 seg <= seg_temp;
154
155 -- Processo de refresh do display --
156 process(clk)
157 begin
158     if rising_edge(clk) then
159         if refresh_cnt >= 100000 then refresh_cnt<=0; active_digit<=active_digit+1;
160         else refresh_cnt<=refresh_cnt+1; end if;
161     end if;
162 end process;
163 process(active_digit, an_temp) begin
164     case active_digit is
165         when 0 => an <= "1110";
166         when 1 => an <= "1101";
167         when others => an <= "1111";
168     end case;
169 end process;
170
171 end Behavioral;

```

5.2 Arquivo de Restrições (nexys3.ucf)

Listing 2: Arquivo de pinagem.

```
1 NET "clk" LOC = "V10" | IOSTANDARD = "LVCMOS33";
2 NET "sw_night" LOC = "T10" | IOSTANDARD = "LVCMOS33";
3 NET "btn_ped" LOC = "B8" | IOSTANDARD = "LVCMOS33";
4
5 # LEDs Rua Secundaria (Direita)
6 NET "leds<0>" LOC = "U16" | IOSTANDARD = "LVCMOS33"; # G
7 NET "leds<1>" LOC = "V16" | IOSTANDARD = "LVCMOS33"; # Y
8 NET "leds<2>" LOC = "U15" | IOSTANDARD = "LVCMOS33"; # R
9
10 # LEDs Rua Principal (Esquerda)
11 NET "leds<3>" LOC = "V15" | IOSTANDARD = "LVCMOS33"; # Off
12 NET "leds<4>" LOC = "M11" | IOSTANDARD = "LVCMOS33"; # Off
13 NET "leds<5>" LOC = "N11" | IOSTANDARD = "LVCMOS33"; # G
14 NET "leds<6>" LOC = "R11" | IOSTANDARD = "LVCMOS33"; # Y
15 NET "leds<7>" LOC = "T11" | IOSTANDARD = "LVCMOS33"; # R
16
17 # Display 7 Segmentos e Anodos (Mapeamento padr o Nexys 3)
18 NET "seg<0>" LOC = "T17" | IOSTANDARD = "LVCMOS33";
19 # ... (demais pinos de seg e an) ...
```

6 Conclusão

O projeto demonstra a aplicação prática de Máquinas de Estados Finitos para resolver problemas de controle de tráfego. O implementai com a ideia de criar um sistema seguro (com tempos de guarda Vermelho-Vermelho), interativo (resposta a pedestres) e robusto (modo de falha/noturno). A utilização da lógica unificada para os LEDs resolveu problemas de concorrência de sinais (*multiple drivers*), resultando em um hardware estável.